

Sumarização Guiada por Palavras-Chave: Integração do YAKE! com Modelos de Linguagem Locais

Tiago Leandro Oliveira Valente

Universidade da Beira Interior, Departamento de Informática, Covilhã, Portugal

Abstract

A sumarização automática de documentos extensos é uma tarefa central no Processamento de Linguagem Natural (PLN). Este trabalho apresenta o desenvolvimento e a avaliação de um sistema de sumarização abstrativa que combina a extração de palavras-chave não supervisionada do algoritmo YAKE! com um modelo de linguagem de larga escala (LLM) executado localmente (*Ollama*, modelo *mistral*). O sistema opera totalmente *offline*, sem dependência de APIs externas. Através de um estudo de ablação e de uma análise paramétrica sistemática, avaliamos a contribuição de cada componente e o impacto de *top_k*, *max_ngram_size* e temperatura na qualidade dos resumos. As métricas automáticas (cobertura de palavras-chave, precisão/*recall* de alinhamento, rácio de compressão) são complementadas por uma avaliação qualitativa através de um juiz LLM nas dimensões de fidelidade, cobertura, coerência e relevância das palavras-chave. Os resultados mostram que o *pipeline* completo (texto + palavras-chave) obtém a melhor pontuação global (4,5 numa escala de 1–5), superando o modo apenas texto (4,19) e o modo apenas palavras-chave (4,13). A configuração *top_k* = 15, *max_ngram_size* = 3 e *temperatura* = 0.5 revelou-se a mais equilibrada, combinando boa cobertura (0,65) e precisão (0,45) sem perda de fidelidade factual.

Palavras-Chave

Sumarização automática, YAKE!, Modelos de linguagem, Ablação, Processamento de Linguagem Natural

1. Introdução

O volume de informação textual produzido diariamente — artigos científicos, notícias, relatórios técnicos, publicações em redes sociais — cresceu de forma exponencial nas últimas décadas, tornando impraticável a leitura e categorização manual destes conteúdos. A sumarização automática, em particular a sumarização abstrativa (geração de novo texto que capta a essência do original), beneficia significativamente da utilização de palavras-chave como guia para o modelo gerador [1]. As palavras-chave identificam os conceitos centrais do documento; o modelo de linguagem pode usá-las como âncora semântica, evitando derivações e melhorando a factualidade do resumo.

Neste trabalho, desenvolvemos um sistema de sumarização abstrativa que integra o extrator de palavras-chave YAKE! [2, 3] com um modelo de linguagem de larga escala (LLM) executado localmente através do *Ollama* (modelo *mistral*). O sistema é completamente *offline*, não requerendo quaisquer chamadas a APIs externas, o que garante a privacidade dos documentos processados e a ausência de custos de utilização.

A principal contribuição deste trabalho é uma análise sistemática da contribuição de cada componente do *pipeline* através de um estudo de ablação. Para tal, definimos três modos de operação:

1. **Pipeline completo** (*full*): fornece ao LLM tanto as palavras-chave extraídas pelo YAKE! como o texto original.
2. **Apenas texto** (*no_keywords*): omite as palavras-chave, fornecendo unicamente o documento original.
3. **Apenas palavras-chave** (*keywords_only*): fornece exclusivamente a lista de palavras-chave, sem o texto original.

Complementarmente, realizamos um estudo paramétrico para determinar a influência de três hiperparâmetros: o número de palavras-chave (*top_k*), o tamanho máximo dos n-gramas (*max_ngram_size*)

UBI 2025/26, Mestrado em Engenharia Informática, Computação em Linguagem Natural

✉ tiago.leandro.valente@ubi.pt (T. L. O. Valente)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

e a temperatura do LLM. A qualidade dos resumos é avaliada através de métricas automáticas (cobertura de palavras-chave, precisão/*recall* de alinhamento, rácio de compressão) e por um modelo de juiz (LLM) que atribui pontuações em quatro dimensões qualitativas.

O artigo está organizado da seguinte forma: a Secção 2 descreve a arquitetura do sistema. A Secção 3 detalha o desenho do estudo de ablação e do estudo paramétrico. A Secção 4 apresenta a configuração experimental. A Secção 5 expõe os resultados obtidos. Finalmente, a Secção 7 resume as conclusões e aponta trabalhos futuros.

2. Arquitetura do Sistema

O sistema desenvolvido segue uma arquitetura *pipeline* modular, composta por quatro componentes principais: (i) normalização e pré-processamento do texto, (ii) extração de palavras-chave com o algoritmo YAKE!, (iii) construção dinâmica de um *prompt* para o modelo de linguagem, e (iv) geração do resumo através de um modelo de linguagem de grandes dimensões (LLM) executado localmente. Adicionalmente, o sistema integra um módulo de avaliação e um painel de ablação interativo.

2.1. Estrutura da Pipeline e Organização do Projeto

O projeto foi desenvolvido seguindo uma organização modular, separando os componentes responsáveis pelo processamento da linguagem natural, disponibilização dos serviços, interface gráfica e execução das experiências. Esta separação facilita a manutenção, reutilização dos componentes e isolamento das diferentes fases da *pipeline*.

A estrutura principal do projeto encontra-se organizada da seguinte forma:

- `src/`: contém a implementação principal da *pipeline* de sumarização. Inclui os módulos responsáveis pelo pré-processamento (`preprocessor.py`), extração de palavras-chave (`keyword_extractor.py`), comunicação com modelos de linguagem (`llm_interface.py`), construção dos *prompts* (`prompt_builder.py`) e execução das avaliações experimentais (`evaluation.py`, `evaluator.py`).
- `backend/`: implementa a camada de serviço da aplicação através de uma API REST. Esta camada é responsável por receber pedidos da interface, executar a *pipeline* de sumarização e devolver os resultados processados. A estrutura `routers/` separa as diferentes funcionalidades da API, incluindo sumarização, avaliação por juiz LLM, execução de ablação, carregamento de amostras e verificação do estado do sistema.
- `frontend/`: contém a interface gráfica *web* desenvolvida em *React*. Esta camada permite ao utilizador interagir com o sistema, submeter documentos, configurar parâmetros da *pipeline* e visualizar resultados de sumarização, métricas e comparações entre diferentes modos de ablação.
- `data/`: armazena os documentos utilizados nos testes experimentais, incluindo textos de entrada, palavras-chave externas para comparação e conjuntos de avaliação anotados.
- `notebooks/`: contém *notebooks* utilizados durante as fases de exploração e desenvolvimento, nomeadamente para testar o YAKE!, configurar modelos locais e validar o funcionamento completo da *pipeline*.
- `outputs/`: guarda os resultados gerados pelas experiências automáticas, incluindo ficheiros JSON e relatórios em *Markdown* produzidos pelos módulos de execução em lote.
- `models/`: reservado para armazenamento ou integração futura de modelos adicionais.
- `report/`: contém a documentação científica do projeto, incluindo o relatório em $\text{\LaTeX}$ e os elementos gráficos associados.
- `scripts/`: contém os *scripts* de extração de amostras, execução de experiências e geração de relatórios consolidados.

A *pipeline* completa segue o seguinte fluxo de execução:

1. O utilizador fornece um documento através da interface gráfica.
2. O *frontend* envia o pedido para o *backend* através de uma API REST.
3. O *backend* encaminha o texto para a *pipeline* principal localizada em `src/`.
4. O módulo de pré-processamento normaliza o texto recebido.
5. O extrator YAKE! identifica as palavras-chave mais relevantes do documento.
6. O construtor de *prompts* combina o texto original e as palavras-chave extraídas, de acordo com o modo selecionado (`full`, `no_keywords` ou `keywords_only`).
7. O pedido é enviado para o modelo de linguagem local através do *Ollama*.
8. O resumo gerado é avaliado através das métricas automáticas e, quando aplicável, pelo módulo de juiz baseado em LLM.
9. Os resultados finais são devolvidos ao *frontend* e apresentados ao utilizador.

Relativamente às tecnologias utilizadas, foram escolhidas ferramentas que permitem executar todo o sistema localmente, garantindo privacidade dos dados e independência de serviços externos.

A linguagem principal utilizada é **Python**, devido ao seu ecossistema desenvolvido para Processamento de Linguagem Natural e aprendizagem automática. A biblioteca *yake* permite a extração não supervisionada de palavras-chave, enquanto os restantes módulos foram desenvolvidos especificamente para integrar a *pipeline* de sumarização.

O **Ollama** foi utilizado como camada de execução dos modelos de linguagem locais. Esta escolha permite utilizar LLMs sem recorrer a APIs externas, reduzindo custos operacionais e garantindo que os documentos processados permanecem no ambiente local. O modelo principal utilizado foi o *mistral*, escolhido pelo equilíbrio entre capacidade de geração textual e requisitos computacionais.

O *backend* foi implementado com **FastAPI**, uma *framework* Python orientada para a criação de APIs rápidas e tipadas. Esta escolha permite uma comunicação eficiente entre a interface gráfica e a *pipeline*, além de facilitar a organização das diferentes funcionalidades através de *routers* independentes.

A interface gráfica foi desenvolvida utilizando **React** com **Vite**. Esta combinação permite criar uma aplicação *web* modular e responsiva, facilitando a criação de componentes reutilizáveis para visualização de métricas, comparação de resultados e configuração dos parâmetros experimentais.

A gestão das dependências é realizada através de `requirements.txt` no ambiente Python e `package.json` / `package-lock.json` no *frontend*, permitindo a reprodução do ambiente de execução.

Esta arquitetura modular permite separar claramente as responsabilidades de cada componente: o *frontend* gere a interação com o utilizador, o *backend* gere a comunicação e controlo da aplicação, e a camada de processamento contém toda a lógica relacionada com sumarização, avaliação e experimentação.

2.2. Normalização e Pré-processamento

O texto fornecido pelo utilizador é submetido a uma normalização que inclui:

- remoção de caracteres não alfanuméricos redundantes;
- normalização de espaços e quebras de linha;
- conversão para minúsculas (configurável);
- limitação do comprimento máximo (por omissão, 3000 caracteres), para evitar que o contexto exceda a janela de contexto do LLM.

Este pré-processamento é aplicado tanto ao texto original como às palavras-chave extraídas, garantindo consistência nas etapas seguintes.

2.3. Extração de Palavras-Chave com YAKE!

A extração de palavras-chave é realizada pela biblioteca *yake* (versão 0.4.8 ou superior), configurável através dos seguintes parâmetros:

- `top_k`: número de palavras-chave a extrair (padrão 12, intervalo 3–30);
- `max_ngram_size`: tamanho máximo dos n-gramas considerados (padrão 3, intervalo 1–5);
- `language`: língua do documento (padrão pt para português, podendo ser alterada para en, es, etc.);
- `deduplication_threshold`: limiar de similaridade para remoção de duplicados (padrão 0.9).

O YAKE! opera exclusivamente sobre o documento individual, sem necessidade de corpora externos. As palavras-chave resultantes são devolvidas como uma lista ordenada por pontuação (quanto menor a pontuação, maior a relevância). No modo *full* do *pipeline*, estas palavras-chave são fornecidas ao LLM juntamente com o texto original.

2.4. Construção do Prompt

O *prompt* enviado ao LLM é gerado dinamicamente por um construtor parametrizado (`prompt_builder.py`). Existem três variantes principais, correspondentes aos modos de ablação:

1. **Pipeline completo** (`full`): inclui uma lista das palavras-chave extraídas, seguida do texto original, com instruções claras para gerar um resumo conciso e factual.
2. **Apenas texto** (`no_keywords`): omite completamente a lista de palavras-chave, fornecendo apenas o texto original.
3. **Apenas palavras-chave** (`keywords_only`): fornece unicamente a lista de palavras-chave, sem qualquer excerto do texto original.

O *template* utilizado é configurável (`zero_shot`, `structured`, `analytical`, `chain_of_thought`). Neste trabalho utilizou-se exclusivamente o *template zero-shot*, por ser o formato mais neutro: não introduz estrutura adicional no *prompt*, permitindo isolar o efeito das palavras-chave sem confundir os resultados com variações induzidas pelo estilo de instrução. Adicionalmente, o utilizador pode controlar os parâmetros `temperature` (0.0–1.0) e `n_gramsize` (2–4) através das opções avançadas da interface, influenciando o equilíbrio entre criatividade e determinismo do LLM.

2.5. Geração com Modelo de Linguagem Local

O sistema utiliza o *Ollama* como *backend* para execução de modelos de linguagem. O modelo utilizado é definido no ficheiro `config.yaml`; por omissão, recorre ao `mistral` (7 mil milhões de parâmetros), mas o utilizador pode escolher entre `llama3.2`, `llama3` ou `qwen2.5:7b` quer através da interface gráfica, quer editando a configuração. O *Ollama* é executado localmente, sem qualquer dependência de APIs externas, garantindo privacidade e controlo sobre os dados.

Caso o *Ollama* não esteja acessível (por exemplo, serviço não iniciado ou configuração de rede incorreta), o sistema apresenta uma mensagem de erro amigável, mas continua a mostrar as palavras-chave extraídas e o *prompt* gerado, permitindo a depuração.

2.6. Avaliação e Métricas

Após a geração do resumo, o módulo `evaluation.py` calcula as seguintes métricas:

- **Cobertura de palavras-chave**: proporção de palavras-chave originais que ocorrem no resumo.
- **Alinhamento (Precisão/Recall)**: comparação entre as palavras-chave extraídas do resumo (usando novamente YAKE!) e as palavras-chave fonte.
- **Rácio de compressão**: dado por $1 - \frac{\text{número de palavras do resumo}}{\text{número de palavras do documento original}}$. Esta métrica, ausente numa versão inicial, foi posteriormente implementada e propagada a todos os módulos (*backend*, *runners*, *interface*).

Quando existem palavras-chave de ouro (*gold keywords*), é também calculada a precisão e *recall* do YAKE! face a esse padrão, permitindo avaliar a qualidade da extração em documentos com anotação manual.

Table 1

Modos de ablação e respectivas entradas para o LLM

Modo	Palavras-chave	Texto original
full	Sim	Sim
no_keywords	Não	Sim
keywords_only	Sim	Não

2.7. Painel de Ablação e Interface Gráfica

A interface *web*, desenvolvida com *React*, inclui um painel específico para o estudo de ablação (`AblationComparison.jsx`). Este painel permite ao utilizador executar os três modos em paralelo (através de `ThreadPoolExecutor` no *backend*) e visualizar os resultados em cartões e gráficos de barras. Foram corrigidos os seguintes problemas identificados durante o desenvolvimento:

- O rácio de compressão aparecia sempre a zero; passou a ser calculado e exibido corretamente.
- O gráfico utilizava a chave `yake_only` em vez da chave correta `keywords_only`, gerando barras vazias; a correção restabeleceu a visualização dos três modos.
- As métricas de precisão e *recall* do alinhamento não eram mostradas nos cartões; foram adicionadas.
- Valores nulos (`null`) são agora apresentados como “—”, em vez de “0.000”, evitando interpretações erradas.

O painel de ablação deixou de aceitar palavras-chave externas, forçando o uso do YAKE! local, tal como definido no plano original. Isto garante que a comparação entre modos se baseia exclusivamente no mesmo conjunto de palavras-chave, tornando o estudo mais rigoroso.

2.8. Estudo Experimental Automatizado

Para permitir a execução sistemática de experiências, foram desenvolvidos três *runners*:

- `batch_runner.py`: percorre uma grelha de parâmetros (*top_k*, *max_ngram_size*, *temperatura*) e um conjunto de documentos, guardando os resultados em JSON e *Markdown*.
- `ablation_runner.py`: executa os três modos de ablação para cada documento, calculando as mesmas métricas.
- `judge_runner.py`: aplica um modelo de juiz (LLM) aos resumos gerados pela ablação, produzindo pontuações qualitativas.

Um orquestrador (`experiment_runner.py`) coordena estes três passos e gera um relatório consolidado.

3. Desenho do Estudo de Ablação e Experiências

3.1. Modos de Ablação

Foram implementados três modos, conforme descrito na introdução. A Tabela 1 resume a informação fornecida ao LLM em cada modo.

3.2. Estudo Paramétrico

Para avaliar o impacto dos hiperparâmetros no *pipeline* completo, definiu-se uma grelha reduzida mas representativa, tendo em conta as limitações computacionais (cada execução do LLM demora cerca de 35 segundos num computador portátil comum). A grelha resultante é:

- $top_k \in \{10, 15, 20\}$ (excluiu-se 5 por fornecer poucas palavras-chave)
- $max_ngram_size \in \{2, 3, 4\}$ (excluiu-se 5 por ganho marginal)
- $temperatura \in \{0.2, 0.5\}$ (excluiu-se 0.8 por induzir alucinações)

Isto totaliza $3 \times 3 \times 2 = 18$ combinações por documento. O modelo de linguagem utilizado foi o *mistral* (7B) através do *Ollama*, mantido fixo durante toda a experiência para isolar o efeito dos parâmetros.

3.3. Documentos de Teste

Extraíram-se 14 documentos de exemplo da demonstração *online* do YAKE! (<http://yake.inesctec.pt/demo.html>), abrangendo várias línguas (inglês, português, alemão, francês, finlandês, etc.) e domínios (tecnologia, desporto, política, economia). Para o estudo principal foram selecionados 4 documentos representativos:

1. Sample1 (inglês, tecnologia)
2. Sample2 (inglês, tecnologia)
3. Sample3 (inglês, tecnologia)
4. Sample4 (português, desporto)

Esta seleção garante diversidade temática e inclui um documento em português para avaliar o comportamento do sistema em diferentes línguas.

3.4. Avaliação por Juiz (Judge)

Além das métricas automáticas, implementou-se um módulo de avaliação qualitativa baseado num LLM (mesmo modelo *mistral*), que classifica cada resumo em quatro dimensões (1 a 5). Note-se que a utilização do mesmo modelo como gerador e como juiz pode introduzir viés de autoavaliação, limitação que é discutida na Secção 6:

- **Fidelidade:** exatidão factual face ao documento original.
- **Cobertura:** capacidade de abranger as informações principais.
- **Coerência:** estrutura lógica e fluência.
- **Relevância das palavras-chave:** integração adequada das palavras-chave fornecidas.

O juiz gera ainda uma pontuação global (média das quatro). Esta avaliação foi aplicada aos resumos gerados no estudo de ablação (12 execuções).

4. Configuração Experimental

As experiências foram executadas num ambiente *Windows Subsystem for Linux* (WSL) com Python 3.10, utilizando o *Ollama* a correr localmente com o modelo *mistral*. O sistema foi configurado com os seguintes parâmetros base (a menos que variados no estudo):

- $top_k = 12$ (padrão)
- $max_ngram_size = 3$ (padrão)
- $temperatura = 0.2$ (padrão)
- comprimento máximo do texto: 3000 caracteres

Para o estudo de ablação, executaram-se os três modos para cada um dos 4 documentos (total de 12 execuções). Para o estudo paramétrico, as 18 combinações foram aplicadas a cada um dos 4 documentos, perfazendo 72 execuções. O módulo juiz foi aplicado às 12 execuções da ablação.

Table 2

Resultados médios do estudo paramétrico (agregados pelos 4 documentos)

<i>top_k</i>	<i>max_ngram</i>	Temp.	Compressão	Cobertura	Precisão (alinh.)	Recall (alinh.)
10	2	0.2	0.664	0.550	0.400	0.400
10	2	0.5	0.618	0.575	0.400	0.400
10	3	0.2	0.683	0.500	0.200	0.200
10	3	0.5	0.682	0.525	0.225	0.225
10	4	0.2	0.656	0.300	0.125	0.125
10	4	0.5	0.676	0.275	0.125	0.125
15	2	0.2	0.675	0.525	0.350	0.350
15	2	0.5	0.693	0.500	0.275	0.275
15	3	0.2	0.695	0.475	0.250	0.250
15	3	0.5	0.647	0.650	0.450	0.450
15	4	0.2	0.671	0.375	0.200	0.200
15	4	0.5	0.653	0.425	0.225	0.225
20	2	0.2	0.686	0.500	0.375	0.375
20	2	0.5	0.694	0.475	0.275	0.275
20	3	0.2	0.693	0.425	0.250	0.250
20	3	0.5	0.678	0.425	0.250	0.250
20	4	0.2	0.685	0.350	0.175	0.175
20	4	0.5	0.683	0.325	0.175	0.175

Table 3

Resultados médios do estudo de ablação

Modo	Compressão	Cobertura	Precisão (alinh.)	Recall (alinh.)	# Palavras médio
full	0,610	0,487	0,139	0,139	141
no_keywords	0,685	0,278	0,083	0,083	119
keywords_only	0,857	0,444	0,167	0,167	58

5. Resultados

5.1. Estudo Paramétrico

A grelha de parâmetros testada produziu 72 resumos. Os valores médios das métricas por combinação de hiperparâmetros estão resumidos na Tabela 2. Observa-se que a cobertura e a precisão de alinhamento tendem a diminuir com o aumento do tamanho do n-grama, especialmente para $max_ngram_size \geq 4$, indicando que n-gramas mais longos capturam termos demasiado específicos que raramente reaparecem no resumo. A temperatura de 0.5 produz resumos ligeiramente mais diversificados sem perda significativa de fidelidade. A configuração $top_k = 15$, $max_ngram_size = 3$ e $temperatura = 0.5$ apresentou o melhor equilíbrio, com cobertura média de 0,65 e precisão de alinhamento de 0,45.

5.2. Estudo de Ablação

A Tabela 3 apresenta as métricas por modo de ablação (médias dos 4 documentos). O modo full atinge a maior cobertura (0,487) e o melhor rácio de compressão médio (0,610), indicando resumos mais concisos sem perda de informação relevante. O modo keywords_only apresenta a maior compressão (0,857), mas à custa de uma cobertura inferior e de alucinações factuais detetadas pelo juiz (ver secção seguinte). O modo no_keywords fica num patamar intermédio.

5.3. Avaliação do Juiz

O juiz LLM avaliou cada resumo da ablação nas quatro dimensões qualitativas. A Tabela 4 apresenta as pontuações médias para o modo full. A Tabela 5 compara os três modos com base na média global

Table 4

Pontuações médias do juiz para o *pipeline* completo (full)

Dimensão	Pontuação média (1–5)
Fidelidade	4,50
Cobertura	4,50
Coerência	4,63
Relevância das palavras-chave	4,38
Overall	4,50

Table 5

Pontuação média global do juiz por modo de ablação

Modo	Pontuação média (1–5)
full	4,50
no_keywords	4,19
keywords_only	4,13

das quatro dimensões.

Os resultados confirmam que o modo `full` é superior em todas as dimensões qualitativas, destacando-se na coerência (4,63) e fidelidade (4,50). O modo `keywords_only` apresenta uma queda acentuada na cobertura (3,5 no documento `Sample3`) e, em alguns casos, introduz alucinações factuais (no documento `Sample2`, o modelo afirmou que a Genius tinha sido encerrada, o que é factualmente falso). O modo `no_keywords` mantém boa fidelidade, mas perde em relevância das palavras-chave e coerência quando comparado com o `full`.

6. Discussão

Os resultados mostram que a integração de palavras-chave extraídas pelo YAKE! no *prompt* do LLM (modo `full`) produz resumos de qualidade superior, quer nas métricas automáticas quer na avaliação qualitativa. A cobertura de palavras-chave é significativamente mais alta no modo `full` (0,487) do que no modo `no_keywords` (0,278), confirmando que as *keywords* orientam o LLM a incluir os conceitos centrais.

O modo `keywords_only`, apesar de apresentar uma cobertura aparentemente razoável (0,444) e uma precisão de alinhamento mais alta (0,167), é penalizado pelo juiz devido à falta de contexto, o que leva a resumos demasiado curtos e, nalguns casos, a invenções factuais. Este fenómeno evidencia a necessidade de utilizar o texto original em conjunto com as palavras-chave.

O rácio de compressão médio do modo `full` (0,610) é adequado para sumarização abstrativa: os resumos têm aproximadamente 39% do comprimento original, mantendo a essência da informação. Modos com compressão excessiva (`keywords_only`, 0,857) sacrificam detalhe e fidelidade.

A avaliação pelo juiz revelou-se consistente com as métricas automáticas, embora tenha identificado nuances que as métricas de sobreposição literal não captam (e.g., alucinações em `keywords_only`). Recomenda-se, para trabalhos futuros, a utilização de um modelo de juiz diferente do modelo gerador para evitar potenciais vieses de autoavaliação.

7. Conclusão e Trabalhos Futuros

Este artigo apresentou o desenvolvimento e a avaliação de um sistema de sumarização abstrativa que combina o extrator YAKE! com um LLM local (*Ollama*, modelo *Mistral*). Através de um estudo de ablação e de uma análise paramétrica sistemática, concluímos que:

- O *pipeline* completo (texto + palavras-chave) é superior aos modos que omitem as *keywords* ou o texto, obtendo a melhor pontuação global (4,5/5) e a maior cobertura de palavras-chave.
- A configuração *top_k* = 15, *max_ngram_size* = 3 e *temperatura* = 0.5 oferece o melhor compromisso entre concisão, cobertura e fidelidade factual.
- As métricas automáticas de alinhamento devem ser interpretadas com cautela, pois favorecem resumos curtos que repetem literalmente as *keywords*, mesmo quando factualmente incorretos.
- O juiz LLM (usando o mesmo modelo *Mistral*) fornece uma avaliação qualitativa útil, mas poderá beneficiar da utilização de um modelo diferente para evitar viés de autoavaliação.

Como trabalho futuro, pretendemos:

- Alargar o estudo a um conjunto mais vasto de documentos e línguas.
- Comparar o YAKE! com outros extratores de palavras-chave (*KeyBERT*, *TextRank*) no mesmo *pipeline*.
- Experimentar *prompts* que integrem as palavras-chave de forma mais estruturada (e.g., no final do texto, com marcadores especiais).
- Utilizar um modelo de juiz diferente (e.g., qwen2 . 5) para avaliar os resumos de forma independente.
- Implementar uma métrica automática de factualidade baseada em NLI (*natural language inference*).

Declaração de uso de Generative AI

Durante a preparação deste trabalho, o autor utilizou o *Claude* (Anthropic) para apoiar a estruturação do documento, revisão gramatical e sugestões de clarificação de frases. Todo o conteúdo técnico, implementação do sistema, desenho experimental e interpretação dos resultados são da exclusiva responsabilidade do autor, que reviu e editou cuidadosamente o manuscrito final.

Referências

- [1] R. Almeida, R. Campos, A. Jorge, S. Nunes, Indexing Portuguese NLP resources with PT-Pump-Up, in: Proceedings of the 16th International Conference on Computational Processing of Portuguese (PROPOR'24), Santiago de Compostela, Spain, 2024, pp. 1–10.
- [2] R. Campos, V. Mangaravite, A. Pasquali, A. Jorge, C. Nunes, A. Jatowt, YAKE! collection-independent automatic keyword extractor, in: European Conference on Information Retrieval, Springer, Grenoble, France, 2018, pp. 806–810.
- [3] R. Campos, V. Mangaravite, A. Pasquali, A. Jorge, C. Nunes, A. Jatowt, YAKE! keyword extraction from single documents using multiple local features, *Information Sciences* 509 (2020) 257–289.

A. Código-Fonte e Recursos

O código-fonte completo do sistema está disponível em <https://github.com/tiagolv/YAKE-SUMMARY-NLP>. Os *scripts* de extração de amostras, execução de experiências e geração de relatórios encontram-se nos diretórios `scripts/` e `src/`.